
A12\Code\A12.py

```
# TODO: Benötigte Pakete importieren
# BEGIN SOLUTION
import cvxpy as cp
import numpy as np
from utils import plot_surface
import matplotlib.pyplot as plt
# END SOLUTION

# TODO: Aufgabenteil 2i. 1. Problem lösen.
print('### Aufgabenteil 2i ###\n')
# BEGIN SOLUTION

# initialisieren unser x
x = cp.Variable(2)
x1 = x[0]
x2 = x[1]

# legen Funktion fest, die zu Minimieren ist
obj = cp.Minimize( (x1 - 3/2)**2 + (x2 - 1/2)**4 )

# Definieren nebenbedingungen
nebenbedingungen = [x1 + x2 - 1 <= 0,
                    x1 - x2 - 1 <= 0,
                    -x1 + x2 - 1 <= 0,
                    -x1 - x2 - 1 <= 0]

# lösen das Problem
problem = cp.Problem(obj, nebenbedingungen)
#minimum = problem.solve(verbose = True)
minimum = problem.solve()

print("Verwendete Solver:", problem.solver_stats.solver_name)
print("Optimierungsstatus:", problem.status)
print("ANzahl Iterationen:", problem.solver_stats.num_iters)
print("benötigte Rechenzeit: ", problem.solver_stats.solve_time, "s")
print("Bestimmter Minimierer: x* = ", x.value)
print("Bestimmtes Minimum: f(x*) = ", minimum)

# END SOLUTION

# TODO: Aufgabenteil 2ii. Iterierte bestimmen und visualisieren.
print('\n### Aufgabenteil 2ii ###\n')
# Hinweis: Verwenden Sie die in utils.py gegebene Funktion
# BEGIN SOLUTION
```

```

anzahl_iterationen = int(problem.solver_stats.num_iters)
x_list = []
for i in range(anzahl_iterationen+1):
    opt_loop = problem.solve(max_iter = i)
    x_list.append([x.value[0], x.value[1]])
    print(f"Iteration: {i}, bestimmter Minimierer: [{x.value[0]:.8e},{x.value[1]:.8e}]")
fig = plot_surface(x_list)
plt.show()
# END SOLUTION

# TODO: Aufgabenteil 2iii. Problem mit Solver SCS l sen
print('\n### Aufgabenteil 2iii ###\n')
# BEGIN SOLUTION
#l sen das Problem mit SCS
#minimum = problem.solve(solver=cp.SCS, verbose = True)
minimum = problem.solve(solver=cp.SCS)

print("Verwendete Solver:", problem.solver_stats.solver_name)
print("Optimierungsstatus:", problem.status)
print("ANzahl Iterationen:", problem.solver_stats.num_iters)
print("ben tigte Rechenzeit: ", problem.solver_stats.solve_time, "s")
print("Bestimmter Minimierer: x* = ", x.value)
print("Bestimmtes Minimum: f(x*) = ", minimum)

print("SCS ist mehr als doppelt so schnell wie CLARABEL, ben tigt jedoch mehr als 12 mal so viele Iterati
(wird durch Abbruchbedingung gestoppt), der bestimmte Minimierer und das Minimum unterscheiden sich
In beiden F llen wird die L sung als optimal angegeben. Allerdings wird auc eine Warnung ausgegeb
'Solution may be inaccurate.'")
# END SOLUTION

# TODO: Aufgabenteil 3. Ottos Optimierungsproblem l sen
print('\n### Aufgabenteil 3 ###\n')
# BEGIN SOLUTION
n = 2
# initialisieren unser x und einen Hilfsvektor
x_otto = cp.Variable(n+1)
zeros_5 = np.zeros(n+1)
zeros_5[-1] = -5

# Definieren zu minimierende Funktion
obj_otto = cp.Minimize(cp.sum_squares(x_otto-zeros_5)-5)
# Definieren nebenbedingungen
nebenbedingungen_otto = [cp.norm(x_otto[:-1]) + x_otto[-1]/5 <= 0,
                          2 * cp.sum_squares(x_otto[:-1]) - 7 - x_otto[-1] <= 0]

#l sen das Problem
problem_otto = cp.Problem(obj_otto, nebenbedingungen_otto)
#problem_otto.solve(verbose = True)
problem_otto.solve()

#l sen das Problem f r verschiedene n
n_werte = [1,3,100000]
for n in n_werte:
    x_otto = cp.Variable(n+1)
    zeros_n = np.zeros(n+1)
    zeros_n[-1] = -5

```

```

obj_otto = cp.Minimize(cp.sum_squares(x_otto-zeros_n)-5)
nebenbedingungen_otto = [cp.norm(x_otto[:-1]) + x_otto[-1]/5 <= 0,
                          2 * cp.sum_squares(x_otto[:-1]) - 7 - x_otto[-1] <= 0]
problem_otto = cp.Problem(obj_otto, nebenbedingungen_otto)
minimum = problem_otto.solve()
print(f"n={n}, Anzahl Iterationen: {problem.solver_stats.num_iters} , benötigte Rechenzeit: {problem.solve_time}")
# END SOLUTION

# TODO: Aufgabenteil 4. Letztes Problem nicht lösen :D
print('\n### Aufgabenteil 4 ###\n')
# BEGIN SOLUTION
#definieren analog wie vorher
x_4 = cp.Variable(2)
obj_4 = cp.Minimize(-(x[0]+1)**2-(x[1]+1)**2)
nebenbedingungen_4 = [cp.sum_squares(x) - 2 <= 0,
                      x[0] - 2**(1/2) <= 0]
problem_4 = cp.Problem(obj_4, nebenbedingungen_4)
# problem_4.solve() #gibt Fehlermeldung "Funktion ist konkav"
print("Das Lösen schlägt Fehl, da die Funktion unter den Nebenbedingungen Konkav ist.") #????????????????
# END SOLUTION

#TERMINAL OUTPUT:

```

Aufgabenteil 2i

Verwendete Solver: CLARABEL

Optimierungsstatus: optimal

Anzahl Iterationen: 12

benötigte Rechenzeit: 0.0006692 s

Bestimmter Minimierer: $x^* = [9.99999997e-01 \ 1.64338071e-09]$

Bestimmtes Minimum: $f(x^*) = 0.31250000231133496$

Aufgabenteil 2ii

Iteration: 0, bestimmter Minimierer: $[5.00000000e-01, 2.90565879e-01]$

Iteration: 1, bestimmter Minimierer: $[6.55128296e-01, 2.24461192e-01]$

Iteration: 2, bestimmter Minimierer: $[9.23929416e-01, 3.07713705e-02]$

Iteration: 3, bestimmter Minimierer: $[9.86512982e-01, 8.17280390e-03]$

Iteration: 4, bestimmter Minimierer: $[9.97176390e-01, 1.88113234e-03]$

Iteration: 5, bestimmter Minimierer: $[9.99664643e-01, 2.15963058e-04]$

Iteration: 6, bestimmter Minimierer: $[9.99962392e-01, 2.05974165e-05]$

Iteration: 7, bestimmter Minimierer: $[9.99993919e-01, 3.34424120e-06]$

Iteration: 8, bestimmter Minimierer: $[9.99998664e-01, 7.22025573e-07]$

Iteration: 9, bestimmter Minimierer: $[9.99999746e-01, 1.37021436e-07]$

Iteration: 10, bestimmter Minimierer: $[9.99999938e-01, 3.27962936e-08]$

Iteration: 11, bestimmter Minimierer: $[9.99999987e-01, 7.01284138e-09]$

Iteration: 12, bestimmter Minimierer: $[9.99999997e-01, 1.64347451e-09]$

Aufgabenteil 2iii

Verwendete Solver: SCS

Optimierungsstatus: optimal

ANzahl Iterationen: 150

benötigte Rechenzeit: 0.0004491999999999997 s

Bestimmter Minimierer: $x^* = [9.99999841e-01 \ 1.14562233e-06]$

Bestimmtes Minimum: $f(x^*) = 0.3124995866623451$

SCS ist mehr als doppelt so schnell wie CLARABEL, benötigt jedoch mehr als 12 mal so viele Iterationen

(wird durch Abbruchbedingung gestoppt), der bestimmte Minimierer und das Minimum unterscheiden sich unwesentlich.

In beiden Fällen wird die Lösung als optimal angegeben. Allerdings wird auch eine Warnung ausgegeben:

'Solution may be inaccurate.'

Aufgabenteil 3

$n=1$, Anzahl Iterationen: 150 , benötigte Rechenzeit: 0.0004491999999999997s, Minimierer: $[0. \ -5.]$, Minimum: -5.0

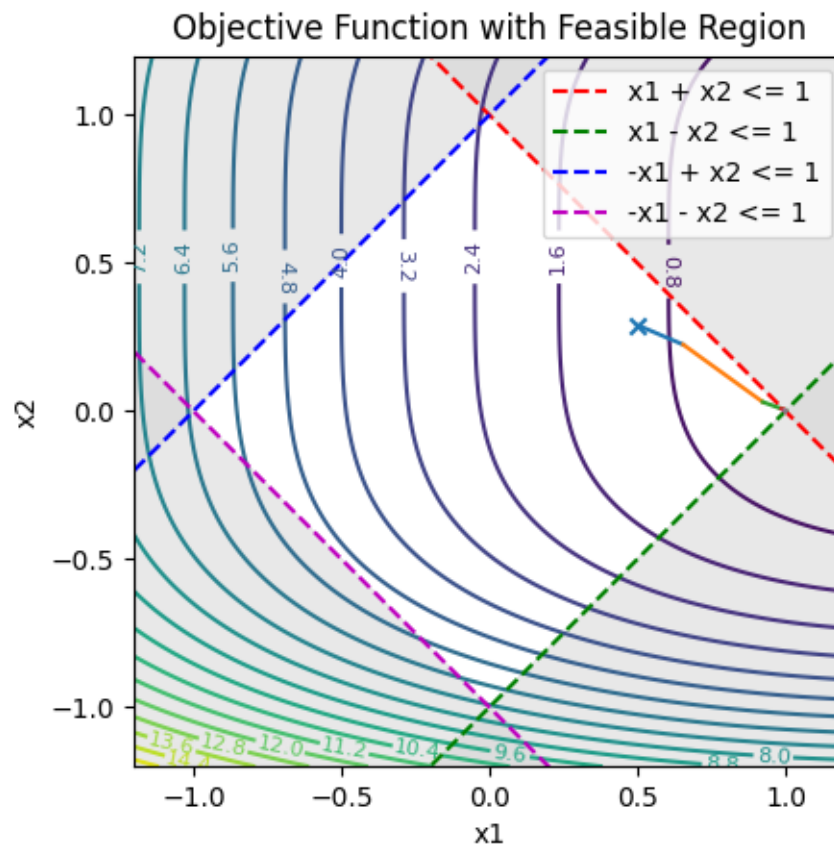
$n=3$, Anzahl Iterationen: 150 , benötigte Rechenzeit: 0.0004491999999999997s, Minimierer: $[0. \ 0. \ 0. \ -5.]$, Minimum: -5.0

$n=100000$, Anzahl Iterationen: 150 , benötigte Rechenzeit: 0.0004491999999999997s, Minimierer: $[0. \ 0. \ 0. \ \dots \ 0. \ 0. \ -5.]$, Minimum: -5.0

Aufgabenteil 4

Das Lösen schlägt fehl, da die Funktion unter den Nebenbedingungen Konkav ist.

#PLOTS:



```
import numpy as np
import matplotlib.pyplot as plt

def plot_surface(xlist=None):
    # Create a grid of x1 and x2 values
    x = np.linspace(-1.2, 1.2, 400)
    X1, X2 = np.meshgrid(x, x)

    # Define the objective function
    Z = (X1 - 3/2)**2 + (X2 - 1/2)**4

    # Define constraints
    c1 = X1 + X2 - 1 <= 0
    c2 = X1 - X2 - 1 <= 0
    c3 = -X1 + X2 - 1 <= 0
    c4 = -X1 - X2 - 1 <= 0

    # Initialise Plot
    fig, ax = plt.subplots()

    # Plot the objective function contours
    contours = ax.contour(X1, X2, Z, levels=20, cmap='viridis')
    ax.clabel(contours, inline=True, fontsize=8)

    # Plot constraint boundaries
    ax.plot(x, 1 - x, 'r--', label='x1 + x2 <= 1')
    ax.plot(x, x - 1, 'g--', label='x1 - x2 <= 1')
    ax.plot(x, x + 1, 'b--', label='-x1 + x2 <= 1')
    ax.plot(x, -x - 1, 'm--', label='-x1 - x2 <= 1')

    # Shade non-feasible regions
    ax.contourf(X1, X2, ~c1, levels=[0.5, 1], colors=['lightgray'], alpha=0.5)
    ax.contourf(X1, X2, ~c2, levels=[0.5, 1], colors=['lightgray'], alpha=0.5)
    ax.contourf(X1, X2, ~c3, levels=[0.5, 1], colors=['lightgray'], alpha=0.5)
    ax.contourf(X1, X2, ~c4, levels=[0.5, 1], colors=['lightgray'], alpha=0.5)

    # Plot iteration process
    if xlist is not None:
        ax.scatter(xlist[0][0], xlist[0][1], marker='x')
        for k in range(len(xlist) - 1):
            xk = xlist[k]
            xkplus1 = xlist[k + 1]
            ax.plot([xk[0], xkplus1[0]], [xk[1], xkplus1[1]])

    # Set aspect ratio and limits
    ax.set_aspect('equal')
    ax.set_xlim(-1.2, 1.2)
    ax.set_ylim(-1.2, 1.2)

    # Labels and legend
    ax.set_xlabel('x1')
    ax.set_ylabel('x2')
    ax.set_title('Objective Function with Feasible Region')
```

```
ax.legend(loc='upper right')

return fig

#TERMINAL OUTPUT:

#PLOTS:
```